

Project Technical Report

16.S897: Spacecraft Attitude Determination and Control Systems

Chase Dunaway

GitHub

github.com/ChaseDunaway21/16-S897

Changelog

Version	Date	Author	Changes
v1.0	2026-04-14	Chase Dunaway	Initial draft. Homework 1 submission. This is the Introduction, the Spacecraft model definition/config, the simulator dynamcis, and Euler's Equation.
v2.0	2026-05-09	Chase Dunaway	Homework 2 submission. This includes the Safe-Mode, implementation of the gyrostat into the simulator, sensor models, and Wahba's Problem. Momentum sphere plots were updated to also show energy gradient.

I Introduction

ARGUS is a 1U CubeSat designed to demonstrate on-board orbit determination using images of Earth's surface captured by the spacecraft¹. It is planned for launch in the Summer of 2026.

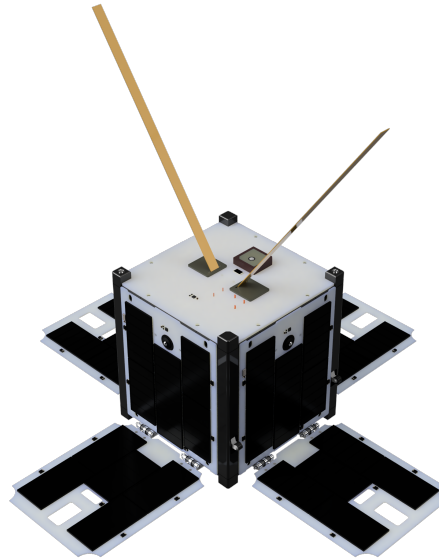


Fig. 1: ARGUS CAD model.

I-A Mission Purpose

The primary purpose of ARGUS is to demonstrate that a 1U CubeSat using minimal, off-the-shelf components, can solve the lost-in-space problem using images of the Earth captured by the satellite in orbit [1]. GPS used in orbit or ground-based radar can be prohibitively expensive for low-cost missions. Moreover, bodies which have been mapped but do not have existing positioning systems such as the moon could utilize the vision-based orbit determination system as ARGUS.

I-B Orbit Description

ARGUS is planned for launch in Summer 2026 and deployment into a sun-synchronous low Earth orbit with an altitude of approximately 600 km and an inclination of approximately 100°.

I-C Mission Requirements and Objectives

Throughout the mission, ARGUS shall:

- 1) Detumble at a residual body rate of < 10 deg/s.

¹Much of the following report is heavily motivated by the implementation details of the ongoing ARGUS development simulator.

- 2) Achieve spin-stabilization < 15 deg of the major inertia axis while spinning < 10 deg/s while pointing within < 10 deg of the Sun direction.
- 3) Acquire and process imagery of the Earth.
- 4) Process images through onboard vision models.
- 5) Cross-validate the position against GPS to within 50 km, and attitude to within 15 deg.
- 6) Downlink telemetry and Payload data.

I-D Mission Timeline

At a high level, ARGUS will operate as a state-machine.

- 1) Launch and deployment.
- 2) Detumbling.
- 3) If spin-rate $>$ threshold $\parallel$ Sun-vector axis > 10 deg nominal $\rightarrow$ Spin-Stabilization.
- 4) If batteries $>$ minimum threshold $\rightarrow$ Nominal operations, image collection, model inference, and orbit determination.
- 5) If batteries $<$ minimum threshold $\rightarrow$ Disable all non-essential components and recharge batteries.

I-E Mechanical Structure

For the purposes of this report, ARGUS is modeled as a uniformly dense 1U cube with four deployable panels. The geometry of the main chassis and the deployed panels are included to capture the effect of the panels on the spacecraft mass properties, inertia matrix, and environmental torque model.

I-F Sensors

The sensor suite consists of a 9-axis Bosch BMX160 IMU [2], a SkyTraq S1216F8-GL GNSS receiver, nine OPT4003 photodiode sun sensors [3], and four Arducam (IMX708) Raspberry Pi Camera Module 3 cameras [4] with a 75° field of view. The magnetometer measurements and gyroscope measurements will be used for detumbling, with the gyroscope measurements also being used for orbit determination. The GNSS receiver will provide timing and a positional baseline to compare against. The sun sensors will be used for sun-pointing. Finally, the cameras will be used to take roughly nadir pointing photos of Earth's surface for inference and orbit determination.

I-G Actuators

The attitude control system consists of six magnetorquers arranged in each of the six faces of the main body. These are used for initial detumbling and control.

II Spacecraft Model

II-A ARGUS Model

ARGUS is modeled as a collection a main, uniform-density cube and 4 rigid components defined in the configuration file. For each component i , the configuration specifies a mass m_i , a geometric center $\mathbf{r}_i \in \mathbb{R}^3$, and a set of exposed faces with dimensions and outward normals.

II-B Discretization & Configuration File

The spacecraft model, initial conditions, and simulation settings are defined in a YAML configuration file. The approximate model and config file (units are in SI) are shown below, with the origin being specified in the center of the main body:

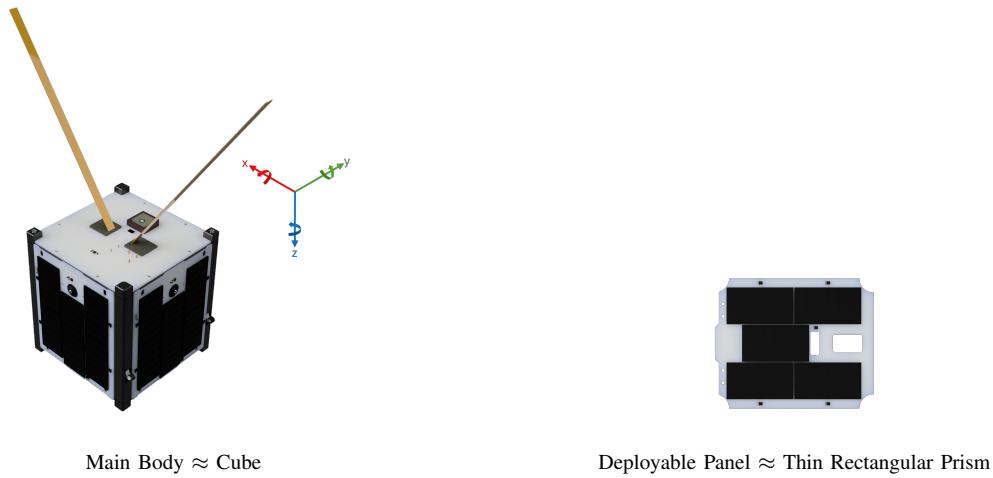


Fig. 2: Two major discretized components used to define the spacecraft model.



Fig. 3: Discretized Components

<pre> physical_properties: - name: main_body dimensions: x: 0.1 y: 0.1 z: 0.1 faces: face_dimensions: x+: [0.1, 0.1] x-: [0.1, 0.1] y+: [0.1, 0.1] y-: [0.1, 0.1] z+: [0.1, 0.1] z-: [0.1, 0.1] face_normals: x+: [1.0, 0.0, 0.0] x-: [-1.0, 0.0, 0.0] y+: [0.0, 1.0, 0.0] y-: [0.0, -1.0, 0.0] z+: [0.0, 0.0, 1.0] z-: [0.0, 0.0, -1.0] geometric_center: x: 0.0 y: 0.0 z: 0.0 mass: 1.15 mass_deviation: 0.05 </pre>	<pre> - name: x+_deployable dimensions: x: 0.1 y: 0.082 z: 0.00385 faces: face_dimensions: z+: [0.1, 0.082] z-: [0.1, 0.082] face_normals: z+: [1.0, 0.0, 0.0] z-: [-1.0, 0.0, 0.0] geometric_center: x: 0.1 y: 0.0 z: -0.498075 normal_vector: x: 1.0 y: 0.0 z: 0.0 mass: 0.03507 mass_deviation: 0.01 - name: x-_deployable dimensions: x: 0.1 y: 0.082 z: 0.00385 faces: face_dimensions: z+: [0.1, 0.082] z-: [0.1, 0.082] face_normals: z+: [1.0, 0.0, 0.0] z-: [-1.0, 0.0, 0.0] geometric_center: x: -0.1 y: 0.0 z: -0.498075 normal_vector: x: -1.0 y: 0.0 z: 0.0 mass: 0.03507 mass_deviation: 0.01 </pre>	<pre> - name: y+_deployable dimensions: x: 0.082 y: 0.1 z: 0.00385 faces: face_dimensions: z+: [0.082, 0.1] z-: [0.082, 0.1] face_normals: z+: [0.0, 1.0, 0.0] z-: [0.0, -1.0, 0.0] geometric_center: x: 0.0 y: 0.1 z: -0.498075 normal_vector: x: 0.0 y: 1.0 z: 0.0 mass: 0.03507 mass_deviation: 0.01 - name: y-_deployable dimensions: x: 0.082 y: 0.1 z: 0.00385 faces: face_dimensions: z+: [0.082, 0.1] z-: [0.082, 0.1] face_normals: z+: [0.0, 1.0, 0.0] z-: [0.0, -1.0, 0.0] geometric_center: x: 0.0 y: -0.1 z: -0.498075 normal_vector: x: 0.0 y: -1.0 z: 0.0 mass: 0.03507 mass_deviation: 0.01 </pre>
---	---	---

Fig. 4: Current YAML configuration for the ARGUS model.

II-C Moment of Inertia

ARGUS's total mass is the sum of the main body and four deployables:

$$M = \sum_{i=1}^N m_i. \quad (1)$$

The center of mass is determined by:

$$\mathbf{r}_{\text{C.o.M.}} = \frac{1}{M} \sum_{i=1}^N m_i \mathbf{r}_i \quad (2)$$

Finally, the inertia tensor is given by:

$$\mathbf{J}_{\text{local}} = \sum_{i=1}^N m_i (\mathbf{r}_i^T \mathbf{r}_i \mathbf{I} - \mathbf{r}_i \mathbf{r}_i^T) \quad (3)$$

In the methodology used to configure ARGUS in the simulator, the dimensions and position vectors from the origin to the center of mass of each component are used. Thus, it is required to take the extra step to compose the inertias.

$$\mathbf{r}' = \mathbf{r}_i - \mathbf{r}_{C.o.M.} \quad (4)$$

and compute

$$\mathbf{J}_{C.o.M.} = \sum_{j=1}^M J_j + \sum_{k=1}^P m_k \left(\mathbf{r}'_k \mathbf{r}'_k \mathbf{I} - \mathbf{r}'_k \mathbf{r}'_k{}^T \right) \quad (5)$$

For brevity, $\mathbf{J}_{C.o.M.}$ will be simplified as $\mathbf{J}$.

Note: Under ideal modeling assumptions, ARGUS is completely symmetric about the z -axis. To make perturbations (and the later momentum-sphere trajectories) visible, a small mass deviation is intentionally introduced; this can be seen in the configuration file.

The following moment of inertia matrices were calculated for the five main discretized components

$$\mathbf{J}_{\text{main body}} = \begin{bmatrix} 5.77 \times 10^{-3} & 5.49 \times 10^{-7} & -3.37 \times 10^{-5} \\ 5.49 \times 10^{-7} & 5.77 \times 10^{-3} & 6.27 \times 10^{-5} \\ -3.37 \times 10^{-5} & 6.27 \times 10^{-5} & 1.92 \times 10^{-3} \end{bmatrix} \quad [\text{kg m}^2] \quad (6)$$

$$\mathbf{J}_{x+, \text{depl.}} = \begin{bmatrix} 9.01 \times 10^{-3} & -4.34 \times 10^{-6} & -2.03 \times 10^{-3} \\ -4.34 \times 10^{-6} & 9.48 \times 10^{-3} & -1.92 \times 10^{-5} \\ -2.03 \times 10^{-3} & -1.92 \times 10^{-5} & 5.23 \times 10^{-4} \end{bmatrix} \quad \mathbf{J}_{y+, \text{depl.}} = \begin{bmatrix} 5.42 \times 10^{-3} & 1.35 \times 10^{-6} & 5.90 \times 10^{-6} \\ 1.35 \times 10^{-6} & 5.15 \times 10^{-3} & -1.18 \times 10^{-3} \\ 5.90 \times 10^{-6} & -1.18 \times 10^{-3} & 3.07 \times 10^{-4} \end{bmatrix}$$

$$\mathbf{J}_{x-, \text{depl.}} = \begin{bmatrix} 7.73 \times 10^{-3} & 3.76 \times 10^{-6} & 1.76 \times 10^{-3} \\ 3.76 \times 10^{-6} & 8.14 \times 10^{-3} & -1.65 \times 10^{-5} \\ 1.76 \times 10^{-3} & -1.65 \times 10^{-5} & 4.57 \times 10^{-4} \end{bmatrix} \quad \mathbf{J}_{y-, \text{depl.}} = \begin{bmatrix} 7.93 \times 10^{-3} & -1.94 \times 10^{-6} & 8.63 \times 10^{-6} \\ -1.94 \times 10^{-6} & 7.54 \times 10^{-3} & 1.69 \times 10^{-3} \\ 8.63 \times 10^{-6} & 1.69 \times 10^{-3} & 4.34 \times 10^{-4} \end{bmatrix}$$

[kg m²] [kg m²]

Using equation (5), the total moment of inertia of ARGUS is calculated to be

$$\mathbf{J}_{\text{spacecraft}} = \begin{bmatrix} 3.59 \times 10^{-2} & -6.21 \times 10^{-7} & -2.90 \times 10^{-4} \\ -6.21 \times 10^{-7} & 3.61 \times 10^{-2} & 5.41 \times 10^{-4} \\ -2.90 \times 10^{-4} & 5.41 \times 10^{-4} & 3.64 \times 10^{-3} \end{bmatrix} \quad [\text{kg m}^2] \quad (7)$$

$$\mathbf{J}_{\text{principal}} = \begin{bmatrix} 3.63 \times 10^{-3} & 0 & 0 \\ 0 & 3.59 \times 10^{-2} & 0 \\ 0 & 0 & 3.61 \times 10^{-2} \end{bmatrix} \quad [\text{kg m}^2] \quad (8)$$

$${}^P\mathbf{R}^B = \begin{bmatrix} 9.01 \times 10^{-3} & -1.67 \times 10^{-2} & 1.00 \\ 1.00 & 2.48 \times 10^{-2} & -8.60 \times 10^{-3} \\ 2.46 \times 10^{-2} & -1.00 & -1.69 \times 10^{-2} \end{bmatrix} \quad (9)$$

II-D Principal Axes

The principal axes are the eigenvectors of the inertia tensor, i.e.

$$\mathbf{J}\mathbf{e}_i = \lambda_i \mathbf{e}_i \quad (10)$$

where the eigenvalues λ_i are the principal moments of inertia. In this report, the moments are ordered

$$J_{1,1} \leq J_{2,2} \leq J_{3,3} \quad (11)$$

so that Axis 1 is the minor axis, Axis 2 is the intermediate axis, and Axis 3 is the major axis.

II-E Surface Discretization

Each exposed face is represented by a centroid, an outward unit normal, and an area. This is simple enough for ARGUS since each face can be closely approximated as a rectangle.

The configuration file stores the face normals $\mathbf{n}_{ij}$ and geometric data needed for future use.

III Dynamics

III-A System State

The full spacecraft state is

$$\mathbf{x} = \begin{bmatrix} \mathbf{r} \\ \dot{\mathbf{r}} \\ \mathbf{q} \\ \boldsymbol{\omega} \end{bmatrix} \in \mathbb{R}^{13} \quad (12)$$

where $\mathbf{r}$ is position, $\dot{\mathbf{r}}$ is velocity, $\mathbf{q} = [q_0 \ q_1 \ q_2 \ q_3]^T$ is a scalar-first quaternion, and $\boldsymbol{\omega} = [\omega_x \ \omega_y \ \omega_z]^T$ is the body angular velocity.

III-B Initialization from Orbital Elements

The initial state $\mathbf{r}$ and $\dot{\mathbf{r}}$ is constructed from the orbital elements $(a, e, i, \Omega, \omega, \nu)$ using the methods in [5]

$$p = a(1 - e^2), \quad (13)$$

the perifocal position

$$r = \frac{p}{1 + e \cos \nu}, \quad \mathbf{r}_{PQW} = \begin{bmatrix} r \cos \nu \\ r \sin \nu \\ 0 \end{bmatrix}, \quad (14)$$

and the perifocal velocity

$$\mathbf{v}_{PQW} = \sqrt{\frac{\mu}{p}} \begin{bmatrix} -\sin \nu \\ e + \cos \nu \\ 0 \end{bmatrix} \quad (15)$$

These vectors are rotated into the inertial reference by the following rotation matrix

$$\mathbf{R} = \begin{bmatrix} \cos \Omega \cos \omega - \sin \Omega \sin \omega \cos i & -\cos \Omega \sin \omega - \sin \Omega \cos \omega \cos i & \sin \Omega \sin i \\ \sin \Omega \cos \omega + \cos \Omega \sin \omega \cos i & -\sin \Omega \sin \omega + \cos \Omega \cos \omega \cos i & -\cos \Omega \sin i \\ \sin \omega \sin i & \cos \omega \sin i & \cos i \end{bmatrix} \quad (16)$$

III-C Orbital Dynamics

The translational dynamics are modeled by spherical acceleration plus a J_2 perturbation [5]

$$\dot{\mathbf{r}} = \mathbf{v}, \quad (17)$$

$$\ddot{\mathbf{r}} = \dot{\mathbf{v}} = \mathbf{a}_g(\mathbf{r}) = -\frac{\mu}{\|\mathbf{r}\|^3} \mathbf{r} + \mathbf{a}_{J_2}(\mathbf{r}), \quad (18)$$

where the J_2 perturbation is

$$\mathbf{a}_{J_2} = \frac{3}{2} \frac{J_2 \mu R_E^2}{\|\mathbf{r}\|^5} \begin{bmatrix} x \left(5 \frac{z^2}{\|\mathbf{r}\|^2} - 1 \right) \\ y \left(5 \frac{z^2}{\|\mathbf{r}\|^2} - 1 \right) \\ z \left(5 \frac{z^2}{\|\mathbf{r}\|^2} - 3 \right) \end{bmatrix} \quad (19)$$

III-D Quaternion Kinematics

The attitude kinematics are written using quaternions

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{G}(\mathbf{q}) \boldsymbol{\omega}, \quad (20)$$

Where $\mathbf{G}(\mathbf{q})$ is the attitude quaternion

$$\dot{\mathbf{q}} = \begin{bmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \frac{1}{2} \boldsymbol{\omega} = \frac{1}{2} \mathbf{L}(\mathbf{q}) \mathbf{H} \boldsymbol{\omega} \quad (21)$$

III-E Numerical Integration

The state is propagated with RK4

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\Delta t}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4), \quad (22)$$

with the quaternion being normalized after each iteration $\|\mathbf{q}\| = 1$.

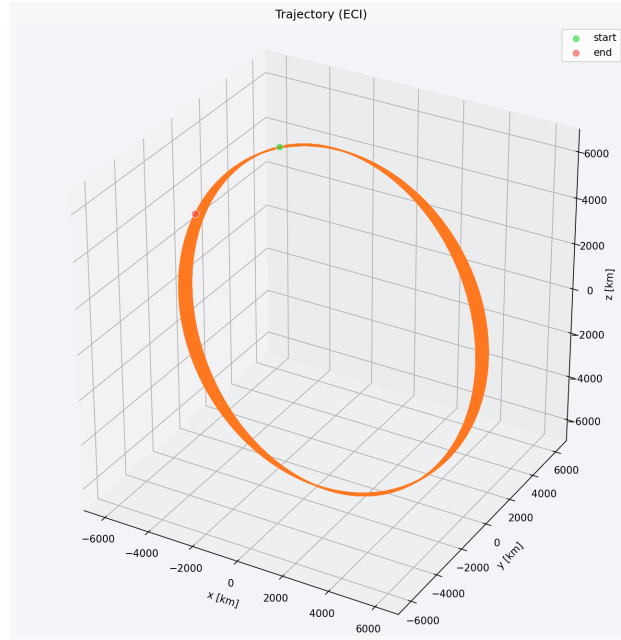


Fig. 5: Orbit trajectory from the dynamics simulation. The simulation was run for 580000 seconds (~ 100 orbits).

IV Euler's Equation

IV-A Rigid-Body Attitude Dynamics

The dynamics for $\dot{\omega}$ are given with Euler's equation in the body frame without external torque

$$\mathbf{J}\dot{\omega} + \omega \times (\mathbf{J}\omega) = \mathbf{0} \quad (23)$$

Which is equivalent to:

$$\dot{\omega} = -\mathbf{J}^{-1} (\omega \times (\mathbf{J}\omega)). \quad (24)$$

IV-B Angular Momentum and Momentum Sphere

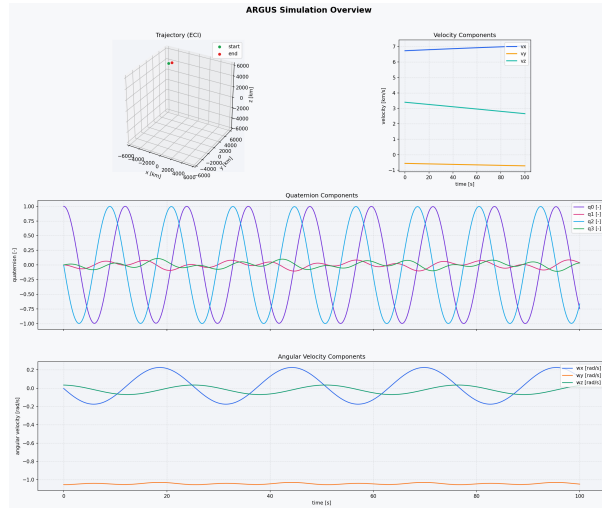
The body angular momentum is

$$\mathbf{h} = \mathbf{J}\omega \quad (25)$$

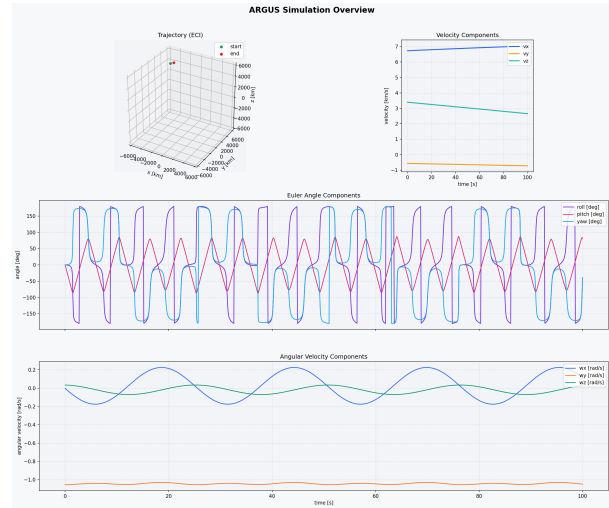
To visualize the momentum sphere, the normalized momentum vector is plotted

$$\hat{\mathbf{h}} = \frac{\mathbf{h}}{\|\mathbf{h}\|} \quad (26)$$

The equilibrium points correspond to rotation about the principal axes, i.e., directions aligned with the eigenvectors of $\mathbf{J}$.

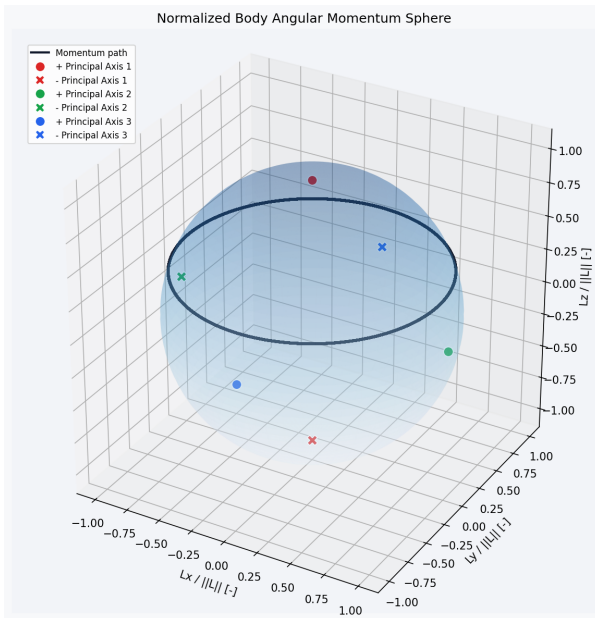


(a) 100 Second simulation plotted with quaternion attitude.

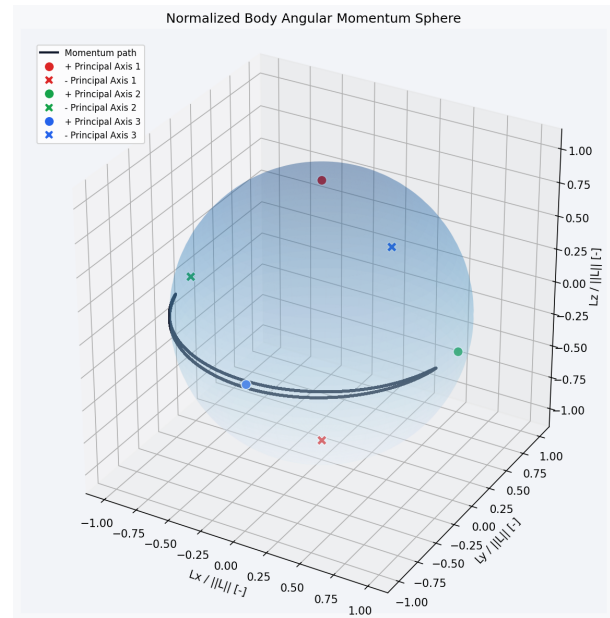


(b) 100 Second simulation plotted with euler-angle attitude.

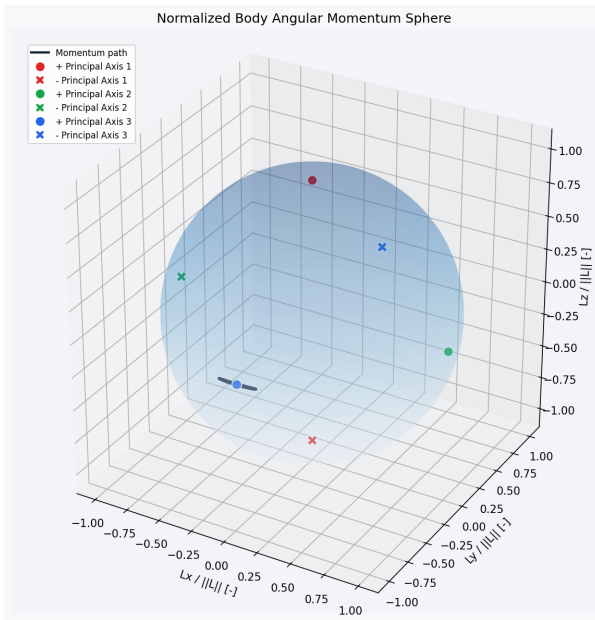
Fig. 6: Attitude history plots from the nominal dynamics simulation.



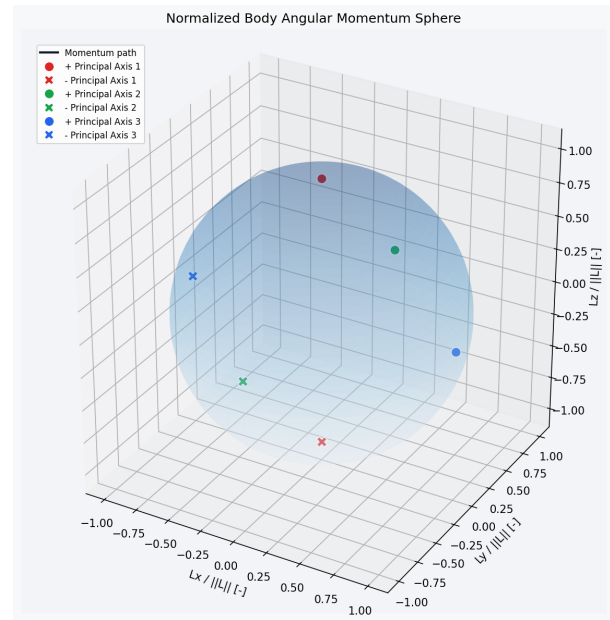
(a) Rotation about inertial axis 1.



(b) Rotation about inertial axis 2.



(c) Rotation about inertial axis 3.



(d) Control (reference) momentum sphere (no perturbation, ideal model assumptions).

Fig. 7: Momentum-sphere trajectories for rotation about each inertial axis, plus the control (reference) case.

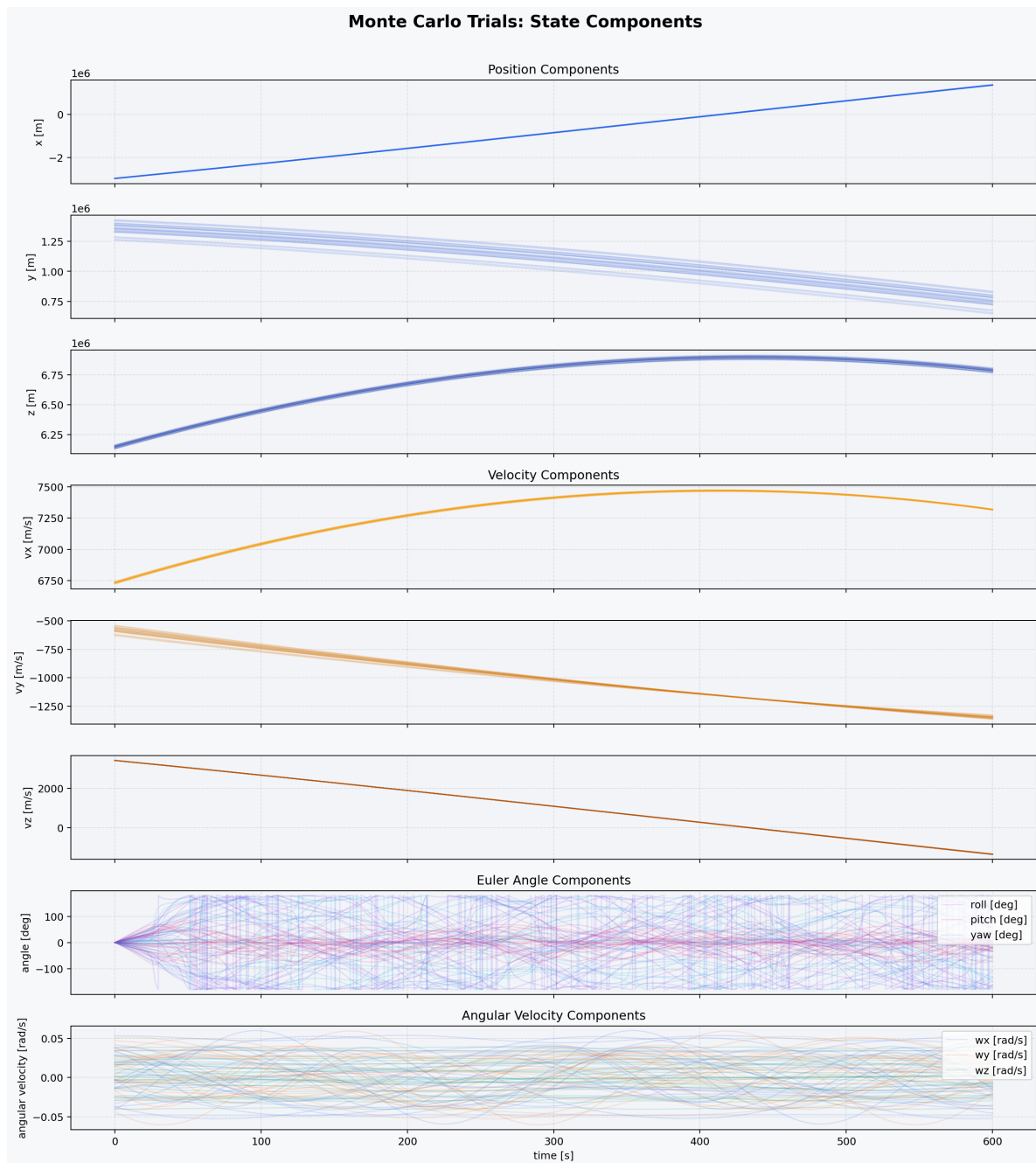


Fig. 8: 25 simulated runs deviated orbital inclinations and initial angular velocities.

V Safe-Mode

For satellites that operate autonomously, a "safe mode" can ensure the satellite obtains an attitude that maximizes its chance to recover while in an inhibited state. For example, a low battery could trigger this state and cause the satellite to attempt to point at the sun while turning off all but its necessary functionality while it recharges.

In the case of this simulated ARGUS, the greater area for solar power generation is by pointing the +z-axis towards the sun. However, in this ARGUS only has magnetorquers, so in this report the simulated satellite differs.

V-A Modified Euler's Equation

First, Euler's equation is modified to include the rotors' angular momentum:

$${}^B h = J^B \omega + {}^B \rho \quad (27)$$

This gives the gyrostat equation

$$\tilde{J}^B \dot{\omega} + {}^B \dot{\rho} + {}^B \omega \times (\tilde{J}^B \omega + {}^B \rho) = {}^B \tau \quad (28)$$

V-B Super Spin

From the lecture notes, for positive nominal spin rate $\omega_0 > 0$, the stability conditions can be written as

$$\omega_0(J_{11} - J_{22}) - \rho_2 < 0 \quad (29)$$

and

$$\omega_0(J_{22} - J_{33}) + \rho_2 > 0 \quad (30)$$

Rearranging these, you get

$$\omega_0 \left[J_{11} - \left(J_{22} + \frac{\rho_2}{\omega_0} \right) \right] < 0 \quad (31)$$

and

$$\omega_0 \left[\left(J_{22} + \frac{\rho_2}{\omega_0} \right) - J_{33} \right] > 0 \quad (32)$$

In both of these cases, the rotor momentum modifies the inertia about the spin axis. In other words, the *effective* spin-axis inertia is

$$J_{\text{eff}} = J_{22} + \frac{\rho_2}{\omega_0} \quad (33)$$

For stability, the effective inertia should be larger than the largest, unmodified principal inertia

$$J_{\text{eff}} \geq 1.2 J_{33} \quad (34)$$

Therefore, for a desired spin rate ω_0 , choose the rotor momentum ρ_2 so that

$$J_{22} + \frac{\rho_2}{\omega_0} = J_{\text{eff}} \quad (35)$$

Solving for the required rotor momentum gives

$$\rho_2 = \omega_0 (J_{\text{eff}} - J_{22}) . \quad (36)$$

V-C Dynamic Balance

For the desired spin to be dynamically balanced, the total angular momentum must be parallel to the angular velocity:

$$\boldsymbol{\omega}_d \times (J\boldsymbol{\omega}_d + \boldsymbol{\rho}) = \mathbf{0} \quad (37)$$

Equivalently,

$$\boldsymbol{\omega}_d^\times \boldsymbol{\rho} = -\boldsymbol{\omega}_d^\times J\boldsymbol{\omega}_d \quad (38)$$

The inertia about the desired spin axis is

$$J_s = \frac{\boldsymbol{\omega}^T}{\omega_s} J \frac{\boldsymbol{\omega}}{\omega_s} \quad (39)$$

The required spin-axis rotor momentum is

$$\rho_s = \omega_s (J_{\text{eff}} - J_s) \quad (40)$$

The rotor momentum must also satisfy the scalar projection constraint

$$\mathbf{s}^T \boldsymbol{\rho} = \rho_s \quad (41)$$

Since $\boldsymbol{\omega}_d = \omega_s \mathbf{s}$, this can be written as

$$\boldsymbol{\omega}_d^T \boldsymbol{\rho} = \omega_s \rho_s \quad (42)$$

Combining the scalar superspin condition with the vector dynamic-balance condition gives the overdetermined linear system

$$\begin{bmatrix} \boldsymbol{\omega}_d^T \\ \boldsymbol{\omega}_d^\times \end{bmatrix} \boldsymbol{\rho} = \begin{bmatrix} \omega_s \rho_s \\ -\boldsymbol{\omega}_d^\times J\boldsymbol{\omega}_d \end{bmatrix} \quad (43)$$

The rotor momentum used in the simulator is the least-squares solution

$$\boldsymbol{\rho} = \arg \min_{\boldsymbol{\rho}} \|A_{\rho} \boldsymbol{\rho} - \mathbf{b}_{\rho}\|_2^2 \quad (44)$$

After adding this to the simulation, I calculated the following $\boldsymbol{\rho}$ vector for 10 RPM around the Safe-Mode axis:

$$\boldsymbol{\rho} = \begin{bmatrix} 0.00312083 \\ -0.00198767 \\ 0.0395601 \end{bmatrix} \text{ [kg m}^2\text{/s]} \quad (45)$$

VI Spacecraft Dynamics

The random perturbations were added as described in the homework.

First, compute the nominal inertia eigen-decomposition

$$J = V D V^T \quad (46)$$

The principal moments are perturbed by a random fractional vector

$$\mathbf{d} \sim \mathcal{N}(0, \Sigma_d) \quad (47)$$

so that

$$\tilde{D} = D(I + \text{diag}(\mathbf{d})) \quad (48)$$

The principal axes are perturbed by a small random rotation vector

$$\boldsymbol{\eta} \sim \mathcal{N}(0, \Sigma_\eta) \quad (49)$$

Using the skew-symmetric matrix

$$\boldsymbol{\eta}^\times = \begin{bmatrix} 0 & -\eta_3 & \eta_2 \\ \eta_3 & 0 & -\eta_1 \\ -\eta_2 & \eta_1 & 0 \end{bmatrix}, \quad (50)$$

the perturbed principal-axis matrix is

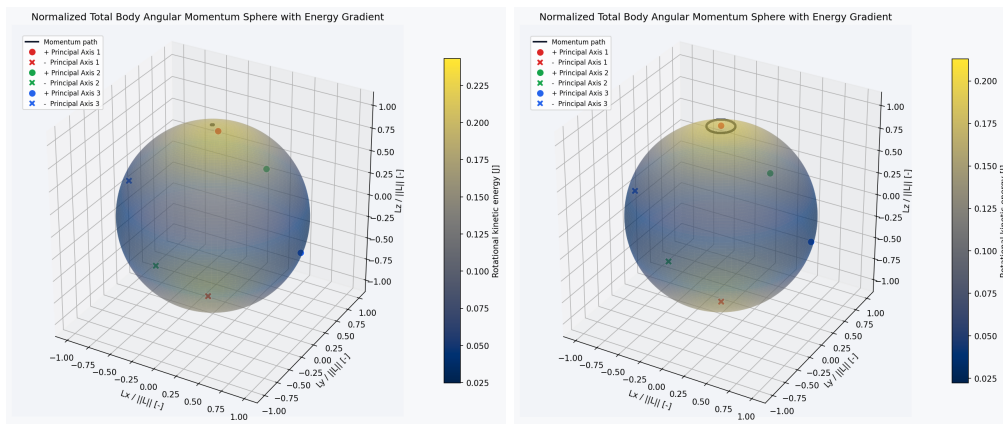
$$\tilde{V} = V \exp(\boldsymbol{\eta}^\times) \quad (51)$$

The perturbed inertia matrix is then

$$\tilde{J} = \tilde{V} \tilde{D} \tilde{V}^T \quad (52)$$

For numerical symmetry, the simulation has this additional step:

$$\tilde{J} \leftarrow \frac{1}{2} (\tilde{J} + \tilde{J}^T) \quad (53)$$



(a) Lower perturbation safe-mode axis.

(b) Higher perturbation safe-mode axis.

Fig. 9: Momentum-sphere rotation around safe-mode.

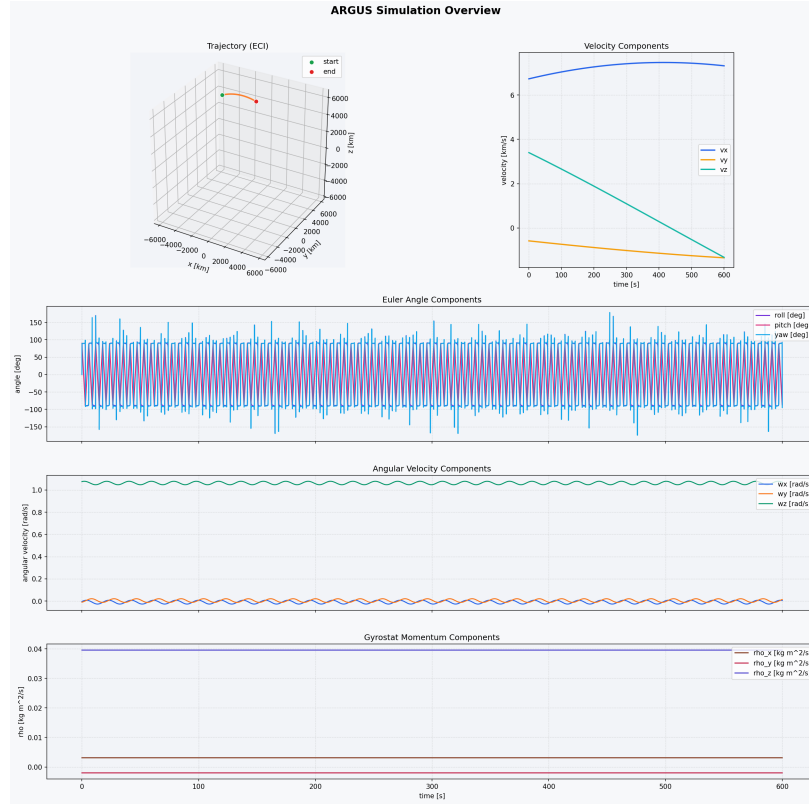


Fig. 10: Safe-Mode 10 Minute Full-Simulation

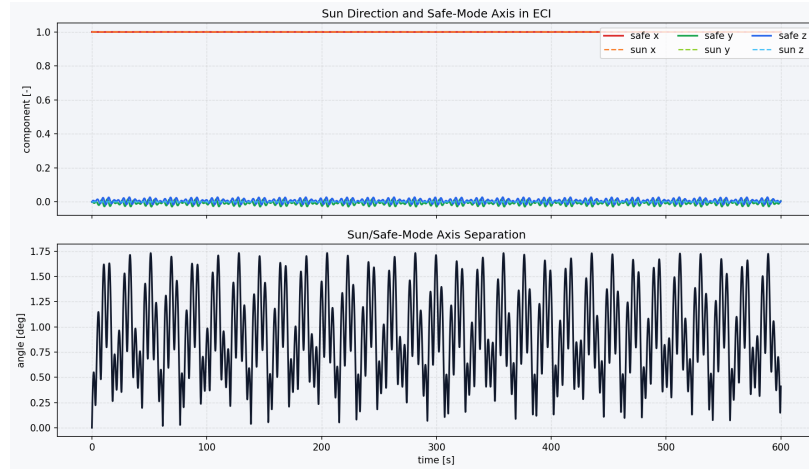


Fig. 11: Safe-Mode Attitude Error

¹The momentum-sphere has a color gradient representing the rotational kinetic energy [5]. For a gyrostat, the total body-frame angular momentum is $\mathbf{h} = J^{-1}(\mathbf{h}_B - \boldsymbol{\rho})$, the rigid-body kinetic energy for a fixed angular-momentum magnitude is

$$T = \frac{1}{2} \mathbf{h}^T J^{-1} \mathbf{h}$$

VII Attitude Sensors

ARGUS uses a Bosch BMX160 IMU [2] that contains an accelerometer, gyroscope, and magnetometer, a set of OPT4003 photodiode sun sensors [3] for sunpointing, and a suite of Arducam (IMX708) visual-band cameras [4] for relative navigation using landmarks.

In order to generate accurate measurements, several models are used for each of the sensors. For all of these models among many other parts of this simulator, the GNC-Simulation repository [6] was a key influence.²

VII-A BMX160 IMU

The Bosch BMX160 is a 9-axis IMU. The provided datasheet [2] shows the following characteristics:

TABLE I: BMX160 accelerometer measurement characteristics

Parameter	Value	Units
Selectable range	$\pm 2, \pm 4, \pm 8, \pm 16$	g
Resolution	16	bit
Sensitivity at $\pm 2g$	16384	LSB/ g
Sensitivity at $\pm 4g$	8192	LSB/ g
Sensitivity at $\pm 8g$	4096	LSB/ g
Sensitivity at $\pm 16g$	2048	LSB/ g
Zero- g offset, component level	± 25	mg
Zero- g offset, board level	± 40	mg
Zero- g offset, lifetime	± 150	mg
Zero- g offset temperature drift	± 1.0	mg/K
Nonlinearity	± 0.5	%FS
Noise density	180	$\mu g/\sqrt{\text{Hz}}$
RMS noise, ODR 200 Hz, 80 Hz filter	1.8	mg-rms
Cross-axis sensitivity	1	%
Alignment error	± 0.5	deg
Output data rate	12.5–1600	Hz

²Many of the simulation tools used for simulating ARGUS have been transferred for this simulator and report, since my original goal was to rewrite the simulator in Python.

TABLE II: BMX160 gyroscope measurement characteristics

Parameter	Value	Units
Selectable range	$\pm 125, \pm 250, \pm 500, \pm 1000, \pm 2000$	deg/s
Sensitivity at ± 2000 deg/s	16.4	LSB/(deg/s)
Sensitivity at ± 1000 deg/s	32.8	LSB/(deg/s)
Sensitivity at ± 500 deg/s	65.6	LSB/(deg/s)
Sensitivity at ± 250 deg/s	131.2	LSB/(deg/s)
Sensitivity at ± 125 deg/s	262.4	LSB/(deg/s)
Zero-rate offset	± 3	deg/s
Zero-rate offset temperature drift	0.05	deg/s/K
Noise density	0.007	deg/s/ $\sqrt{\text{Hz}}$
RMS noise, ODR 200 Hz, 74.6 Hz filter	0.07	deg/s-rms
Bias stability	3	deg/hr
Cross-axis sensitivity	2	%
Output data rate	25–3200	Hz

TABLE III: BMX160 magnetometer measurement characteristics

Parameter	Value	Units
Magnetic field range, x/y axes	± 1300	μT
Magnetic field range, z axis	± 2500	μT
Device resolution	0.3	μT
Gain error after compensation	± 2	%
Sensitivity temperature drift	± 0.01	%/K
Zero- B offset	± 40	μT
Zero- B offset after calibration	± 2	μT
Full-scale nonlinearity	1	%FS
Output noise, low-power preset, $x/y/z$	1.0/1.0/1.4	$\mu\text{T-rms}$
Output noise, regular preset, $x/y/z$	0.6/0.6/0.6	$\mu\text{T-rms}$
Output noise, enhanced regular preset	0.5/0.5/0.5	$\mu\text{T-rms}$
Output noise, high-accuracy preset	0.3/0.3/0.3	$\mu\text{T-rms}$
Recommended ODR, forced mode	12.5	Hz

The accelerometer is completely unused in the simulation, but is configurable.

According to the datasheet, the noise of the gyro in deg/s should have a variance around 0.07^2 or $1.49 * 10^{-6}$ deg/s. Similarly, the magnetometer should have a noise variance of 1.4^2 or $1.96 \mu\text{T}$.

In practice, this is not the case. Both of these numbers are substantially lower than the true measured output my sample BMX160.

To get a better representation, I collected 14 hours worth of data and calculated the actual covariance and bias of the IMU using `allantools`.

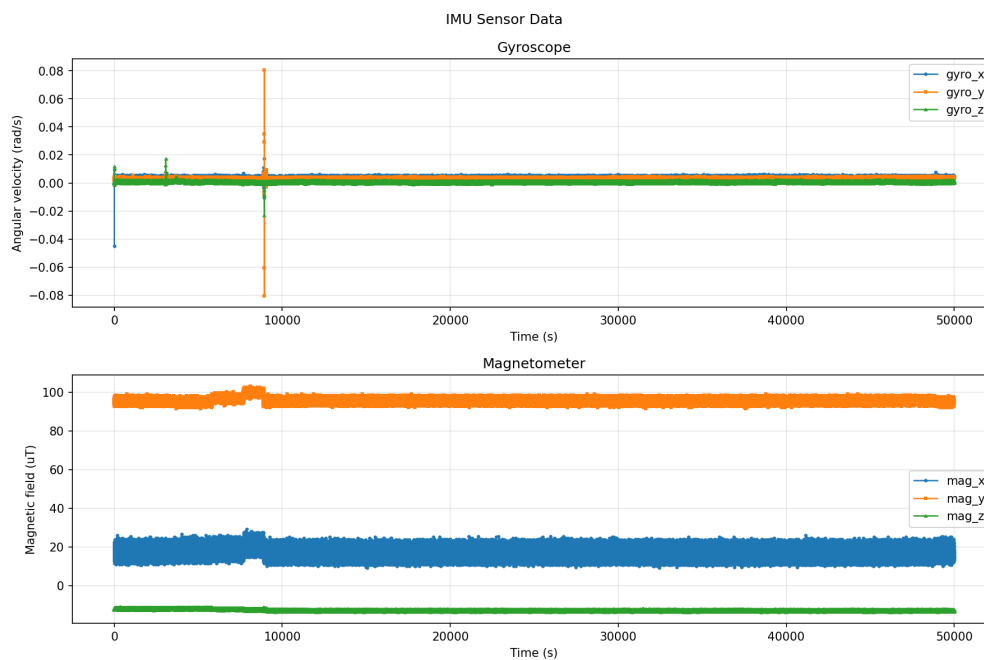


Fig. 12: Raw BMX160 IMU measurement time series used for noise characterization.

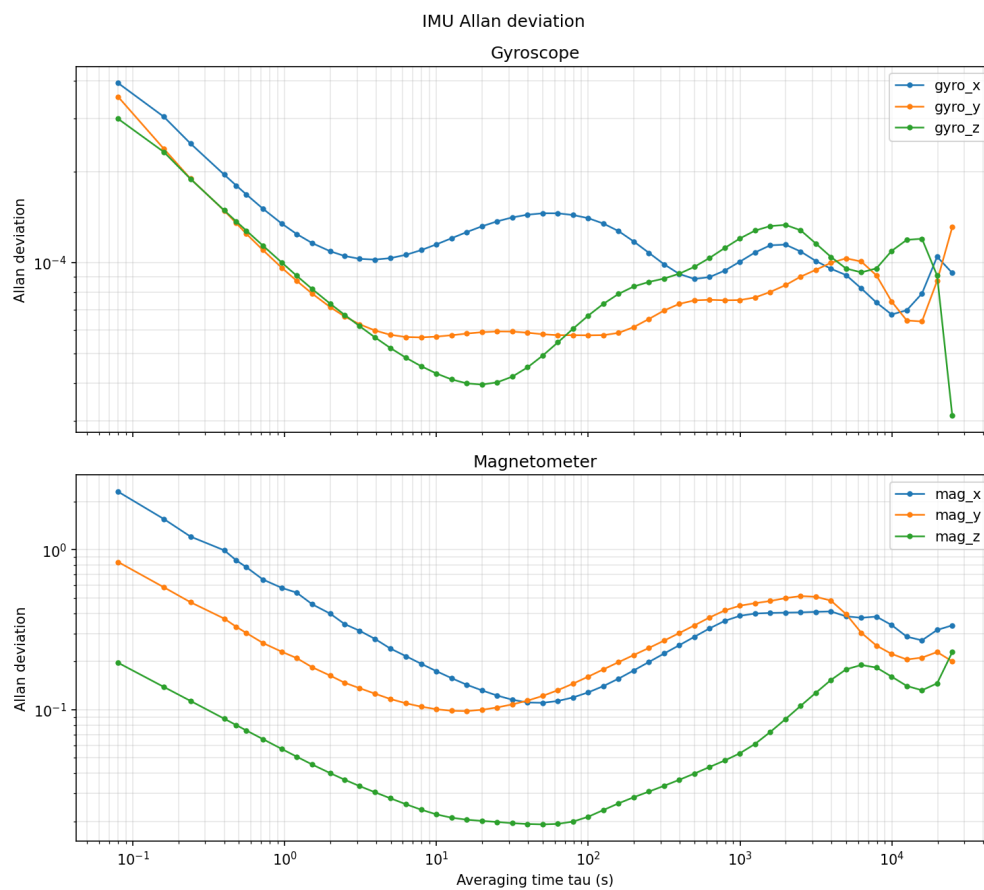


Fig. 13: Allan deviation computed from the BMX160 IMU data using allantools.

The measured covariance and biases was as follows:

Gyroscope

$$f_g = > 25.0 \text{ Hz}, \quad (54)$$

$$b_g = \begin{bmatrix} 0.004 \\ 0.003 \\ 0.0008 \end{bmatrix} \text{ rad/s}, \quad (55)$$

$$R_g = \begin{bmatrix} 2.55133216013 \times 10^{-7} & 1.248810858 \times 10^{-8} & 1.73119113409 \times 10^{-8} \\ 1.248810858 \times 10^{-8} & 1.63402081184 \times 10^{-7} & 5.20736207649 \times 10^{-10} \\ 1.73119113409 \times 10^{-8} & 5.20736207649 \times 10^{-10} & 1.54844505556 \times 10^{-7} \end{bmatrix} (\text{rad/s})^2 \quad (56)$$

Magnetometer

$$f_m = 12.5 \text{ Hz}, \quad (57)$$

$$b_m = \begin{bmatrix} 16.2709815383 \\ 95.4802525636 \\ -12.6296260353 \end{bmatrix} \mu\text{T}, \quad (58)$$

$$R_m = \begin{bmatrix} 5.47678999806 & 1.11960431476 & 0.151622883711 \\ 1.11960431476 & 1.20637910543 & 0.0291750864857 \\ 0.151622883711 & 0.0291750864857 & 0.15363690581 \end{bmatrix} \mu\text{T}^2 \quad (59)$$

While the gyro and magnetometer can update at much higher frequencies, setting the output to 10-5 Hz causes the simulator to slow down dramatically, especially so for the magnetometer since the measurement model (discussed later) samples the IGRF-14 model [7].

Lastly, the bias for the gyro is not constant. The datasheet claims a random walk of 3 deg/hr, which is roughly 1.45×10^{-5} rad/s. This is set as the default random-walk value for all three-axis.

VII-B Magnetic Field Model

The magnetic field model is generated using the IGRF-14 model [7].

VII-C OPT4003 Sun Sensor

In the simulation, the OPT4003 Sun Sensors used on ARGUS are abstracted to instead give a bearing vector output instead of light intensity. A conservative estimate to how it will perform is $\sigma_\theta \approx 5^\circ$. It is set to have an update frequency of 0.1 Hz.

VII-D Sun Model

The Sun model uses JPL/NAIF SPICE (via a Python interface) [8] as well as a Montenbruck–Gill conical shadow model [9] to allow for eclipse effects to be generated.

The Montenbruck–Gill conical shadow model is defined as so:

$\mathbf{r}_E$ is defined as the vector from the Earth center to the spacecraft, and $\mathbf{r}_S$ is defined as the vector from the Sun to the spacecraft. Define

$$r = \|\mathbf{r}_E\|, \quad d = \|\mathbf{r}_S\| \quad (60)$$

The angular radii of the Sun and Earth are

$$a = \sin^{-1} \left(\frac{R_\odot}{d} \right), \quad b = \sin^{-1} \left(\frac{R_E}{r} \right), \quad (61)$$

where $R_\odot$ is the Sun radius and R_E is the Earth radius.

The angular separation between the apparent Sun center and Earth center is

$$c = \cos^{-1} \left(\frac{\mathbf{r}_E^T \mathbf{r}_S}{\|\mathbf{r}_E\| \|\mathbf{r}_S\|} \right) \quad (62)$$

For the penumbra case, define

$$x = \frac{c^2 + a^2 - b^2}{2c} \quad (63)$$

and

$$y = \sqrt{a^2 - x^2} \quad (64)$$

The overlap area between the apparent solar disk and the apparent Earth disk is

$$A_{\text{overlap}} = a^2 \cos^{-1} \left(\frac{x}{a} \right) + b^2 \cos^{-1} \left(\frac{c-x}{b} \right) - cy. \quad (65)$$

The illumination 'fraction' is then

$$\nu = \begin{cases} 0, & r \leq R_E, \\ 1, & a + b \leq c, \\ 0, & c < b - a, \\ 1 - \frac{A_{\text{overlap}}}{\pi a^2}, & \text{otherwise.} \end{cases} \quad (66)$$

VII-E ArduCam 3 Cameras

Similarly to the Sun Sensor, the ArduCam 3 module is abstracted to give a bearing vector output. The role of the cameras is to take photos of landmarks and perform inference using vision models. As a result, the update rate for the cameras are arbitrarily set to 0.01 Hz as it is unreasonable to have a high update rate for these measurements. However, the resolution of the images is high, with the bounding-boxes around many landmarks being small relative to 75° F.o.V. Assuming a 10 km error when nadir pointing, this gives 0.5° pointing accuracy, which is significantly higher than that of the coarse Sun Sensor.

VII-F Earth-Camera Model

The Earth-Image-Capture Model is currently defined in the code as a bearing measurement from the satellite to a target within some accuracy. In the future, targets will be defined in ECEF and measurements will only be taken if the satellite passes overhead. But currently, the model is simply constrained periodically only.

VII-G Bearing-Vector Noise Sampling

To add noise to the bearing-vector measurements, the noise covariance is defined using a simple orthogonal projection of Gaussian, isotropic noise (consistent with standard two-vector attitude-determination measurement models) [10]:

$$W_b = \sigma_\theta^2 (I_3 - \mathbf{b}\mathbf{b}^T) \quad (67)$$

The variance σ_θ^2 is set in the `config.yaml` file.

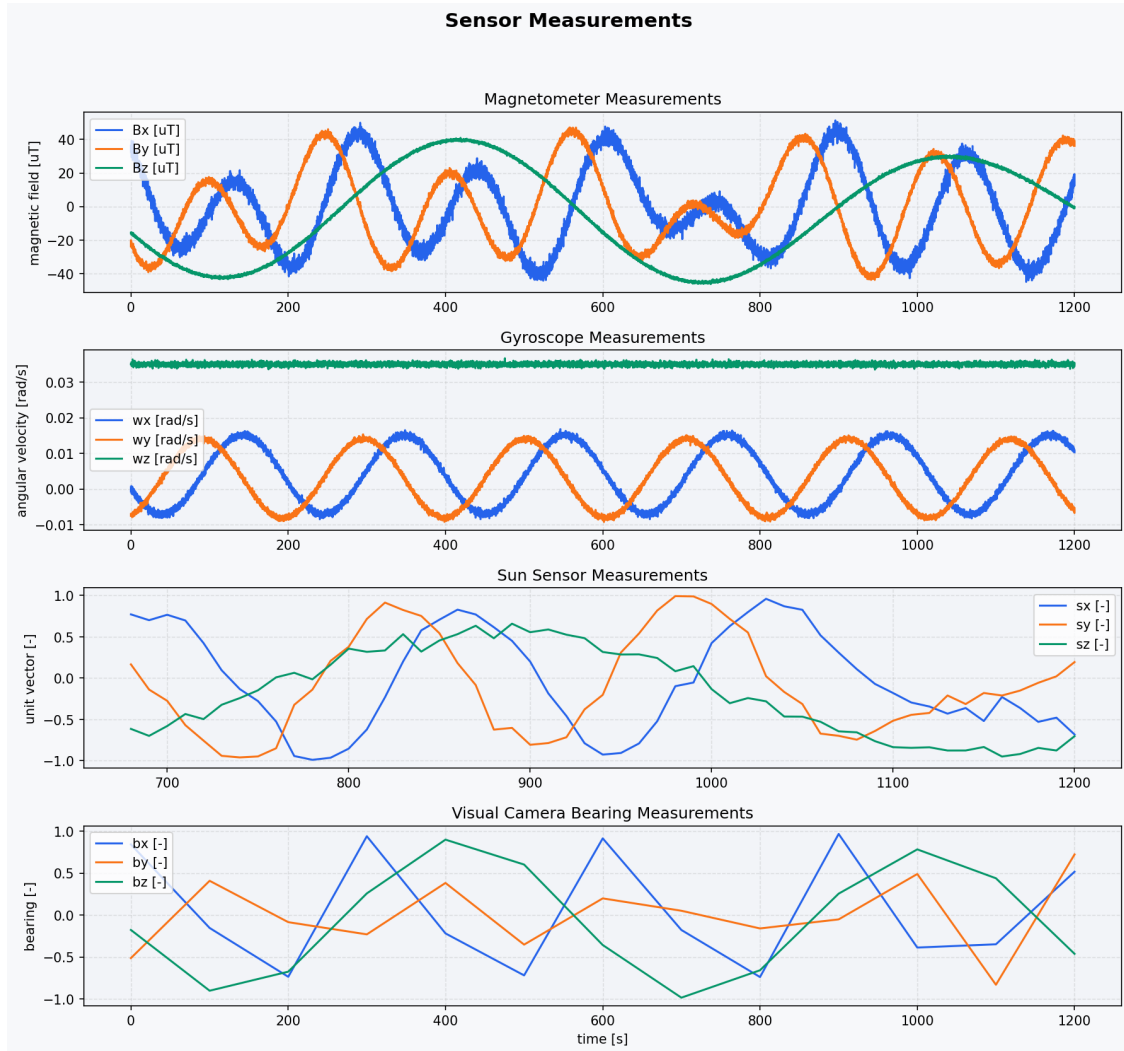


Fig. 14: Sensor measurements over 20 minute simulation.

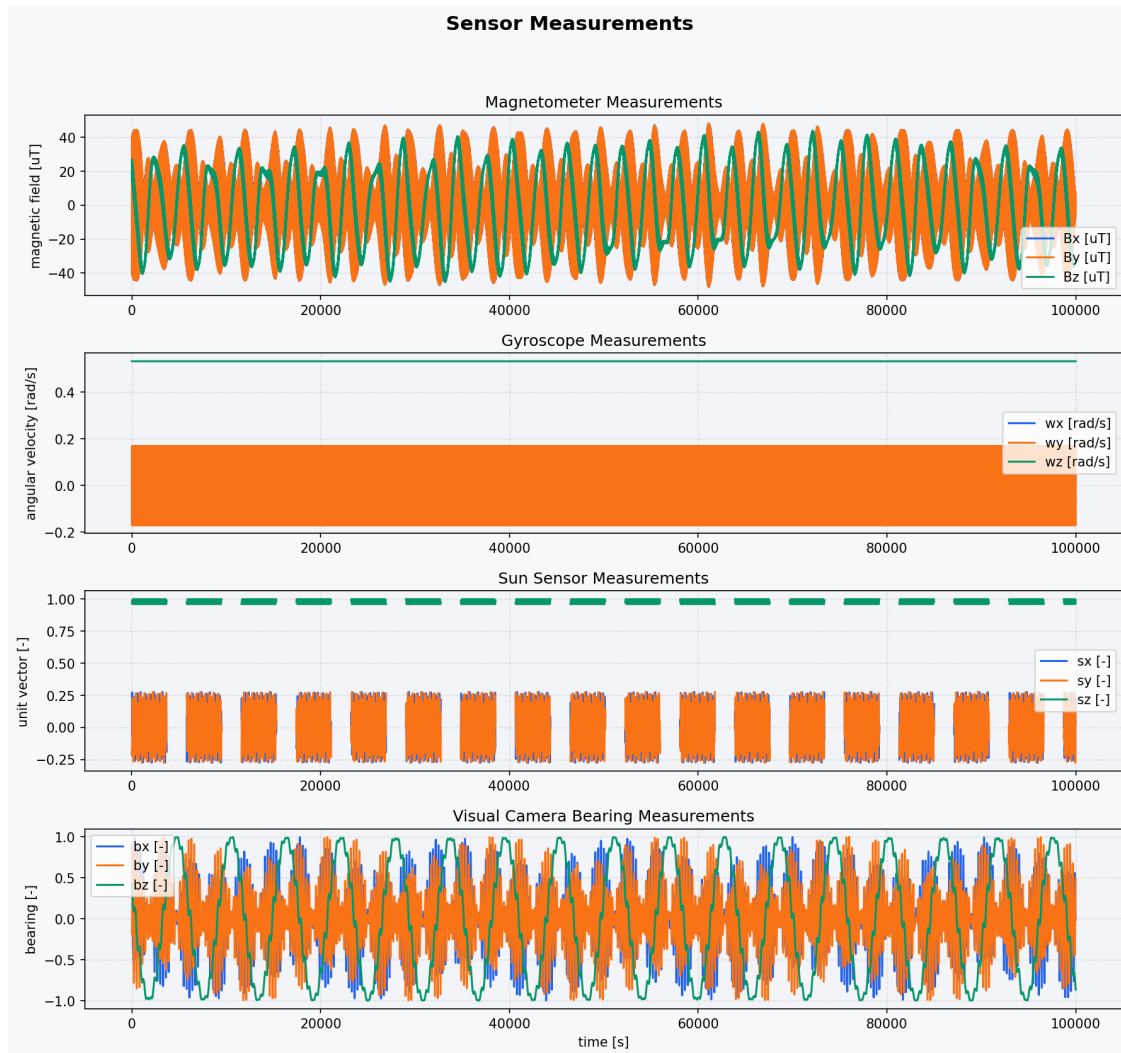


Fig. 15: Sensor measurements over an 18-orbit simulation in safe-mode.

VIII Wahba's Problem

In class, Wahba's Problem was solved three ways: Singular-Value Decomposition (SVD), the q-Method, and Semi-Definite Programming (SDP). In my simulator I have implemented the SVD and SDP methods.

VIII-1 SVD Wahba Solution

Given body-frame measurements $\mathbf{b}_i$ and inertial reference vectors $\mathbf{r}_i$, first normalize each vector:

$$\hat{\mathbf{b}}_i = \frac{\mathbf{b}_i}{\|\mathbf{b}_i\|}, \quad \hat{\mathbf{r}}_i = \frac{\mathbf{r}_i}{\|\mathbf{r}_i\|} \quad (68)$$

Stack the measurements as rows:

$$B_{\text{body}} = \begin{bmatrix} \hat{\mathbf{b}}_1^T \\ \hat{\mathbf{b}}_2^T \\ \vdots \\ \hat{\mathbf{b}}_N^T \end{bmatrix}, \quad R_{\text{ref}} = \begin{bmatrix} \hat{\mathbf{r}}_1^T \\ \hat{\mathbf{r}}_2^T \\ \vdots \\ \hat{\mathbf{r}}_N^T \end{bmatrix} \quad (69)$$

The attitude profile matrix is

$$A = B_{\text{body}}^T R_{\text{ref}} \quad (70)$$

Compute the singular value decomposition

$$A = U \Sigma V^T. \quad (71)$$

Define the determinant correction

$$M = \text{diag}(1, 1, \det(VU^T)) \quad (72)$$

Thus, the SVD Wahba solution is

$$Q_{\text{SVD}} = V T M U^T \quad (73)$$

where Q_{SVD}

$$\hat{\mathbf{b}}_i \approx Q_{\text{SVD}} \hat{\mathbf{r}}_i \quad (74)$$

VIII-2 SDP Wahba Solution

Using the same attitude profile matrix

$$A = B_{\text{body}}^T R_{\text{ref}}, \quad (75)$$

the constrained optimization form used in the simulator is

$$\begin{aligned} Q_{\text{SDP}} = \arg \max_{Q \in \mathbb{R}^{3 \times 3}} \quad & \text{tr}(Q^T A) \\ \text{subject to} \quad & \begin{bmatrix} I_3 & Q^T \\ Q & I_3 \end{bmatrix} \succeq 0 \end{aligned} \quad (76)$$

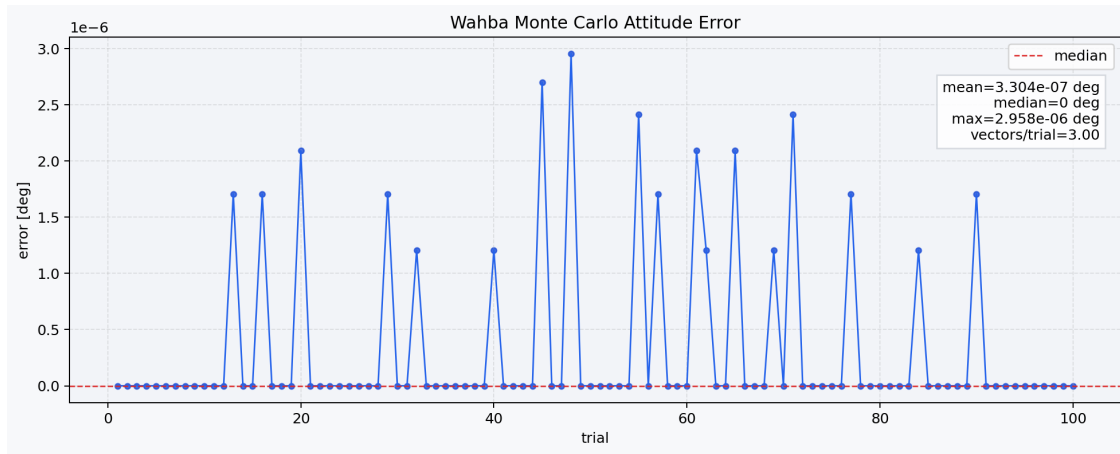
VIII-3 Simulation

ARGUS is configured to have three sensors that output bearing-vector measurements. To simulate solving for attitude given the magnetometer, sun-sensor, and camera, monte-carlo campaigns were run with 100 trials.

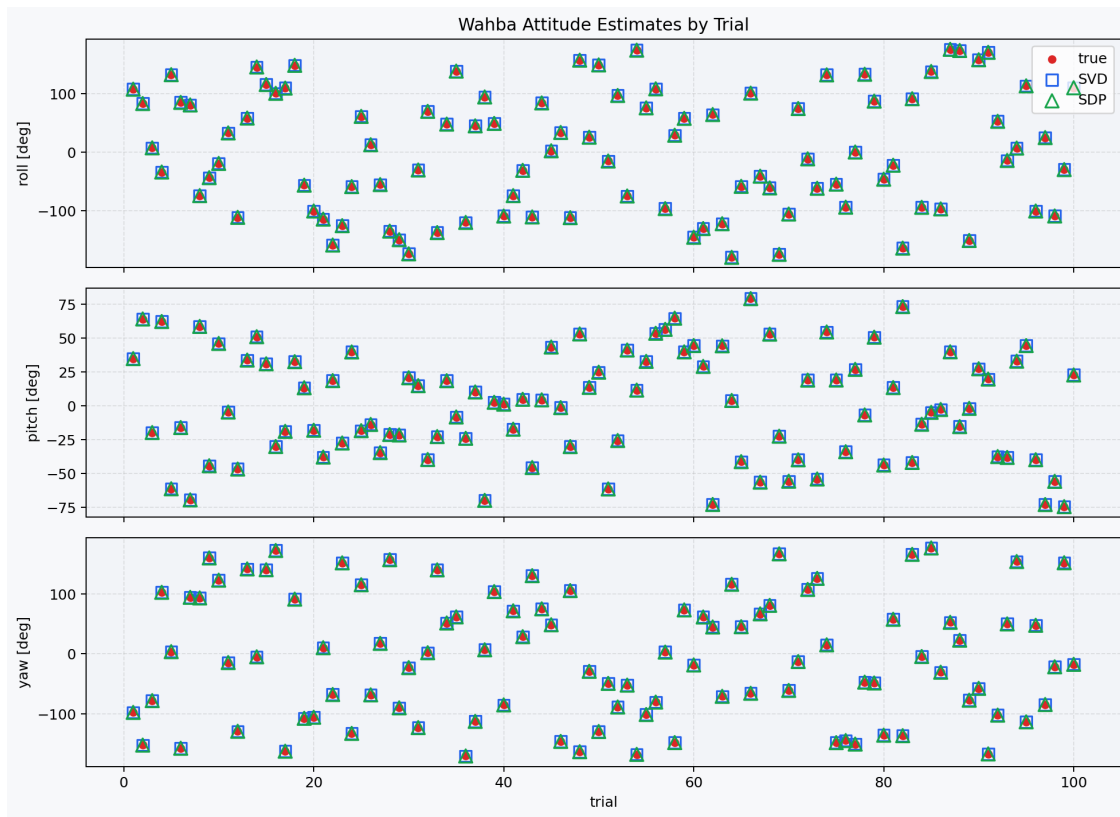
In the first case, the bearing measurements have no noise. Here the SVD, SDP, and truth align closely. In 100-trials, the maximum error was 0.00° and the mean was 0.00° for both the SVD and SDP methods.

In the second case, the measurements have the configured noise. Here the SVD and SDP methods struggle, as the sun-sensor is only accurate to within 5° . Also, in trials where two-measurements are nearly aligned, the solution is significantly further off. In 100-trials, the maximum error was 28.3° and the mean was 10.1° for both the SVD and SDP methods.

Something to note is that the measurements are weighted with identity when creating the B matrix. Weighting the camera measurement significantly higher would help, but even so the variance from the BMX160 and coarse Sun Sensor is still poor for precise attitude determination with only three measurements at any given timestep.

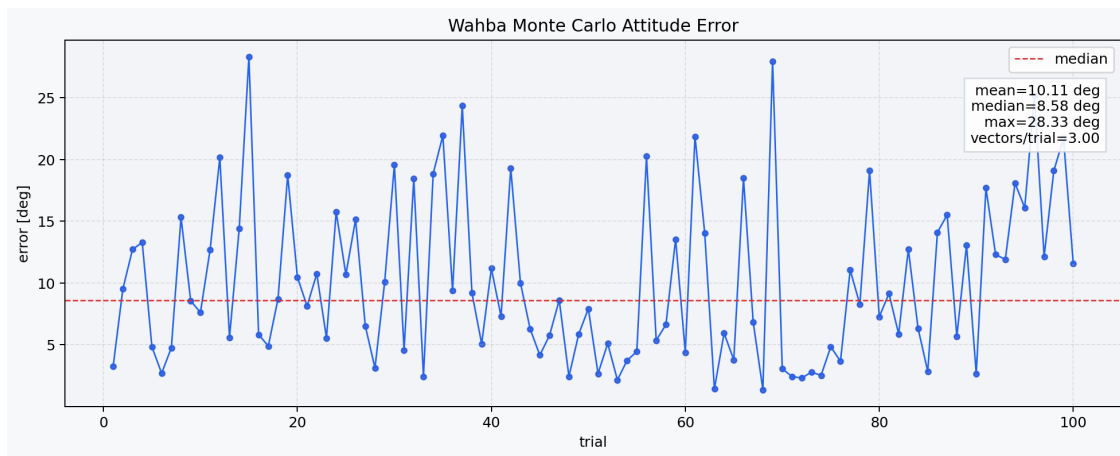


(a) Monte Carlo results (noise-free measurements).

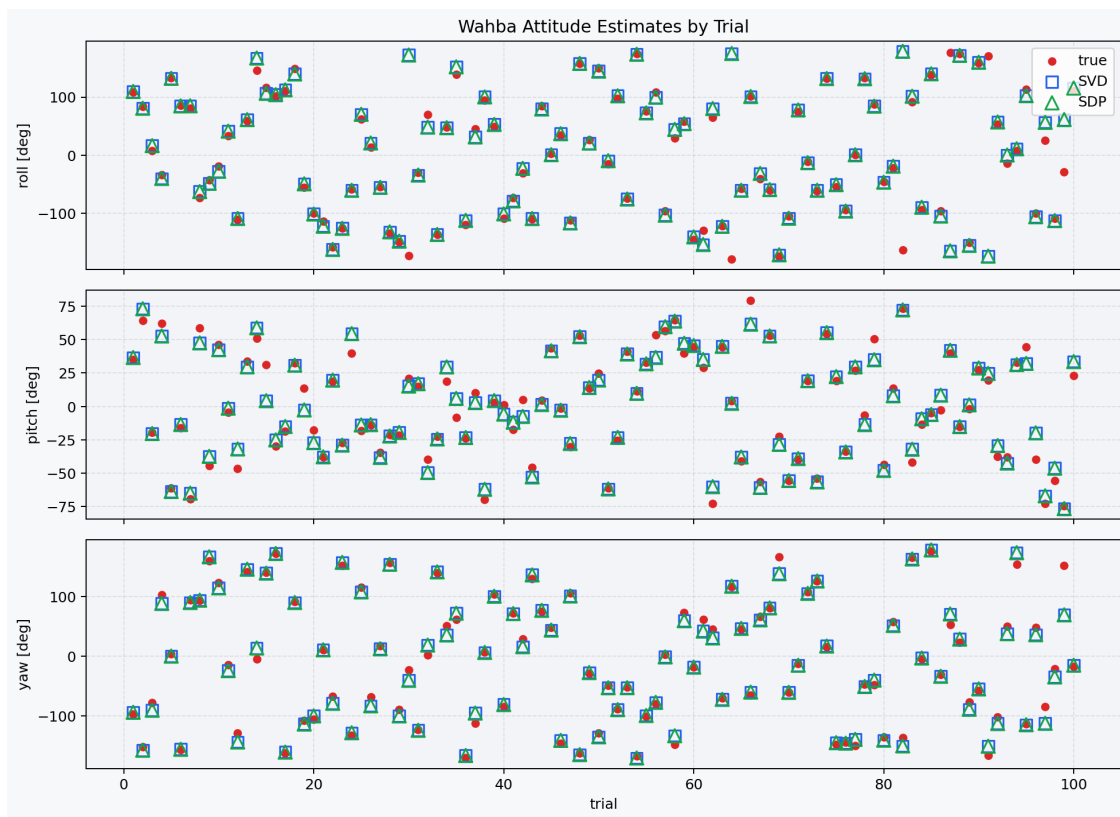


(b) Recovered attitudes (noise-free measurements).

Fig. 16: Wahba Monte Carlo results (noise-free case).



(a) Monte Carlo results (noisy measurements).



(b) Recovered attitudes (noisy measurements).

Fig. 17: Wahba Monte Carlo results (noisy case).

References

- [1] K. McCleary, S. Gurumurthy, P. R. M. Fisch, S. Tayal, Z. Manchester, and B. Lucia, “Vinsat: Solving the lost-in-space problem with visual-inertial navigation,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 11 774–11 780.
- [2] Bosch Sensortec, *BMX160: Small, Low Power 9-Axis Sensor*, Bosch Sensortec GmbH, May 2018, datasheet, revision 1.1. Accessed: 2026-05-07. [Online]. Available: <https://www.mouser.com/pdfdocs/BST-BMX160-DS000-11.pdf>
- [3] *OPT4003-Q1 High-Speed, Dual-Channel, High-Precision, Digital Ambient Light Sensor*, Texas Instruments, Jan. 2026, datasheet, document no. SBASA69E; revised January 2026. [Online]. Available: <https://www.ti.com/lit/ds/symlink/opt4003-q1.pdf>
- [4] *12MP IMX708*, Arducam, 2026, arducam Wiki documentation for Raspberry Pi native camera module. [Online]. Available: <https://docs.arducam.com/Raspberry-Pi-Camera/Native-camera/12MP-IMX708/>
- [5] F. L. Markley and J. L. Crassidis, *Fundamentals of Spacecraft Attitude Determination and Control*, ser. Space Technology Library. New York, NY: Springer, 2014, vol. 33.
- [6] cmu-argus-2, “GNC-Simulation,” GitHub repository, 2024, accessed: 2026-05-09. [Online]. Available: <https://github.com/cmu-argus-2/GNC-Simulation>
- [7] International Association of Geomagnetism and Aeronomy, “IGRF-14,” 2024, version v1. Accessed: 2026-05-09. [Online]. Available: <https://doi.org/10.5281/zenodo.14012303>
- [8] NASA Navigation and Ancillary Information Facility, “Spice toolkit,” Software and documentation, accessed: 2026-05-09. [Online]. Available: <https://naif.jpl.nasa.gov/naif/toolkit.html>
- [9] O. Montenbruck and E. Gill, *Satellite Orbits: Models, Methods and Applications*. Berlin: Springer, 2000.
- [10] F. L. Markley, “Attitude determination using two vector measurements,” NASA Goddard Space Flight Center, Tech. Rep., Sep. 1998, accessed: 2026-05-09. [Online]. Available: <https://ntrs.nasa.gov/citations/19990052720>
- [11] National Geospatial-Intelligence Agency, “Department of defense world geodetic system 1984: Its definition and relationships with local geodetic systems,” National Geospatial-Intelligence Agency, Tech. Rep. NGA.STND.0036_1.0.0_WGS84, 2014, accessed: 2026-05-07. [Online]. Available: https://earth-info.nga.mil/GandG/publications/NGA_STND_0036_1_0_0_WGS84/NGA.STND.0036_1.0.0_WGS84.pdf
- [12] Jet Propulsion Laboratory, “Astrodynamics parameters,” Solar System Dynamics, NASA / California Institute of Technology, accessed: 2026-05-07. [Online]. Available: https://ssd.jpl.nasa.gov/astro_par.html
- [13] R. S. Park, W. M. Folkner, J. G. Williams, and D. H. Boggs, “The jpl planetary and lunar ephemerides de440 and de441,” *The Astronomical Journal*, vol. 161, no. 3, p. 105, 2021.
- [14] A. Prša, P. Harmanec, G. Torres, E. Mamajek, M. Asplund, N. Capitaine, J. Christensen-Dalsgaard, É. Depagne, M. Haberreiter, S. Hekker, J. Hilton, G. Kopp, V. Kostov, D. W. Kurtz, J. Laskar, B. D. Mason, E. F. Milone, M. Montgomery, M. Richards, W. Schmutz, J. Schou, and S. G. Stewart, “Nominal values for selected solar and planetary quantities: Iau 2015 resolution b3,” *The Astronomical Journal*, vol. 152, no. 2, p. 41, 2016.
- [15] United States Naval Observatory, “Terrestrial time (tt),” Astronomical Applications Department, accessed: 2026-05-07. [Online]. Available: <https://aa.usno.navy.mil/faq/TT>
- [16] Astro- und Feinwerktechnik Adlershof GmbH, “Reaction wheel rw1 for small satellites,” Product page, accessed: 2026-05-07. [Online]. Available: <https://www.astrofein.com/en/reaction-wheels/reaction-wheel-rw1/>
- [17] J. S. Subirana, J. M. J. Zornoza, and M. Hernández-Pajares, “Transformations between ecef and enu coordinates,” ESA Navipedia, 2011, accessed: 2026-05-07. [Online]. Available: https://gssc.esa.int/navipedia/index.php/Transformations_between_ECEF_and_ENU_coordinates
- [18] Wikipedia contributors, “Conversion between quaternions and euler angles,” Wikipedia, The Free Encyclopedia, accessed: 2026-05-07. [Online]. Available: https://en.wikipedia.org/wiki/Conversion_between_quaternions_and_Euler_angles
- [19] —, “Rotation matrix: Quaternion,” Wikipedia, The Free Encyclopedia, accessed: 2026-05-07. [Online]. Available: https://en.wikipedia.org/wiki/Rotation_matrix#Quaternion
- [20] I. Harris and W. Priester, “Time-dependent structure of the upper atmosphere,” *Journal of the Atmospheric Sciences*, vol. 19, no. 4, pp. 286–301, 1962.
- [21] NASA Navigation and Ancillary Information Facility, “NAIF0012 Leapseconds Kernel,” SPICE generic kernel, 2016, accessed: 2026-05-09. [Online]. Available: https://naif.jpl.nasa.gov/pub/naif/generic_kernels/lsk/naif0012.tls

- [22] —, “DE442s Planetary Ephemeris SPK Kernel,” SPICE generic kernel, 2025, accessed: 2026-05-09. [Online]. Available: https://naif.jpl.nasa.gov/pub/naif/generic_kernels/spk/planets/de442s.bsp